PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

UE24CS341A — Software Engineering

Software Architecture and Design Specification

for

Student Attendance Management System (SAMS)

Version 1.1 (Pending Approval)

Based on SRS v1.1 and Software Test Plan v1.1

Updated to reflect the work done in Sprints 1 and 2

Prepared by Team 3

Anmol Vyas (PES2UG24CS072)

Anupam Rajesh Tiwari (PES2UG24CS073)

Anupriya (PES2UG24CS074)

Anusha G (PES2UG24CS075)

PES University, Bengaluru

30-10-2026

Revision History

Version	Date	Author	Change Summary
1.0	03-10-2026	Team 3	Initial Software Architecture and Design Specification, based on SRS v1.0 and Software Test Plan v1.0.
1.1	30-10-2026	Anmol Vyas	Updated to reflect the work done in Sprints 1 and 2: open items in 4.6 closed, and a new Section 6 with the interfaces as built, the differences from version 1.0 and why, and the division of work as carried out.

Approvals

Role	Name	Signature / Date
Course Coordinator	Asha Kumari (asha.hebbar@gmail.com)	Pending

Table of Contents

1. Introduction **4**
1.1 Purpose 4
1.2 Scope 4
1.3 Audience 4
1.4 Definitions 4
2. Document Overview **5**
2.1 How to Use This Document 5
2.2 Related Documents 5
3. Architecture **6**
3.1 Goals and Constraints 6
3.2 Stakeholders and Concerns 6
3.3 Component (UML) Diagram 6
3.4 Component Descriptions 7
3.5 Chosen Architecture Pattern and Rationale 8
3.6 Technology Stack and Data Stores 8
3.7 Risks and Mitigations 10
3.8 Traceability to Requirements 10
3.9 Security Architecture 11
4. Design **13**
4.1 Design Overview 13
4.1.1 Data Model 13
4.1.2 Division of Work 14
4.2 UML Sequence Diagrams 16
4.3 API Design 18
4.4 Error Handling, Logging, and Monitoring 19
4.5 UX Design 20
4.6 Open Issues and Next Steps 20
4.7 Architecture Decision Records 21
5. Appendices **23**
5.1 Glossary 23
5.2 References 23
5.3 Tools 23
6. Changes in Version 1.1 ..... **24**
6.1 Interfaces as Built 24
6.2 Other Differences from Version 1.0 24
6.3 Division of Work as Carried Out 25

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Student Attendance Management System (SAMS). It turns the requirements of the SRS (v1.0) into a concrete structure of components, data, interfaces, and behaviour that the team will implement in Python/Django with MySQL, and that the test cases of the Software Test Plan (v1.0) will verify.

1.2 Scope

The design covers every SAMS service defined in the SRS: user authentication with role-based access; student, faculty, and subject management; attendance marking, modification, and duplicate prevention; attendance percentage calculation and low-attendance alerts; attendance history; student-wise, subject-wise, and class-wise reports; and audit logging of administrative actions. Payroll, examination evaluation, fee management, and other non-attendance activities remain out of scope, as in the SRS.

1.3 Audience

Developers, QA engineers, security reviewers, project guides and instructors, and future maintenance teams.

1.4 Definitions

Term	Definition
SAMS	Student Attendance Management System
SRS / STP / RTM	Software Requirements Specification / Software Test Plan / Requirements Traceability Matrix
SAD	Software Architecture and Design Specification (this document)
ADR	Architecture Decision Record: a short record of one significant design decision and its rationale
MVT	Model–View–Template, Django's variant of the MVC pattern
ORM	Object-Relational Mapper (the Django ORM)
RBAC	Role-Based Access Control
STRIDE	Threat categories: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
TLS / HTTPS	Transport Layer Security / HTTP over TLS
CSRF	Cross-Site Request Forgery; Django rejects form posts without a valid CSRF token
CI/CD	Continuous Integration / Continuous Deployment (GitHub Actions workflow in this project)
Deactivation	Setting is_active = false on a student or faculty account instead of deleting it, so attendance history is preserved
Time slot	A defined one-hour class period (for example Period 1, 09:00–10:00). Attendance is recorded per time slot, so a one-hour theory class is one class and a two-hour lab, which occupies two slots, is two classes

2. Document Overview

2.1 How to Use This Document

Section 3 describes the architecture: goals, stakeholders, the UML component diagram, component responsibilities, the architecture pattern, technology stack, risks, traceability to every SRS requirement, and the security architecture. Section 4 describes the detailed design: the data model, the division of work among team members, UML sequence diagrams, API (interface) definitions, error handling, UX guidelines, open issues, and architecture decision records. Section 5 holds the appendices.

2.2 Related Documents

Document	Version	Relationship to this SAD
Software Requirements Specification (SRS), Team 3	v1.1	Source of the 20 functional, 7 non-functional, and 5 security requirements designed here. Its RTM (SRS Section 8) cites design-spec IDs (DS-*) that this document defines in Section 3.8.
Software Test Plan (STP), Team 3	v1.1	Defines the test strategy and test case IDs (TC-*) that verify this design. The open items in STP Section 13.1 are resolved in Section 4.6 of this document.
Project repository and README, Team 3	GitHub, main branch	https://github.com/anu-9t/Student-Attendance-Management-System. Holds the submitted SRS and STP and the project overview, stack, and status. The README is to be updated to match this document (Section 4.6).
SE Mini-Project Guidelines and Deliverables Part-1, PES University	—	Course requirements for this document and the mandated process: Git and GitHub with code reviews, a CI/CD pipeline, Docker, static analysis, security validation, and unit tests with code and branch coverage.

3. Architecture

3.1 Goals and Constraints

Goals

- Integrity of attendance data: no duplicate records and no unauthorised or untraceable changes (SAM-F-013, SO-02).
- Security: authentication before access, strict role separation, student data isolation, hashed passwords, and an audit trail (SAM-SEC-001 to SAM-SEC-005).
- Reliability: at least 99% availability during academic working hours, with regular backup and recovery (SAM-NF-002, SAM-NF-005).
- Performance: 90% of normal requests complete within 3 seconds, with multiple simultaneous users supported (SAM-NF-001, SAM-NF-007).
- Maintainability: new features can be added without major changes to existing modules (SAM-NF-006).

Constraints

- Python 3 with Django and MySQL 8.0 are fixed for the project (SRS Sections 2.1 and 2.5).
- Passwords use a one-way hashing algorithm; plaintext storage is prohibited (SAM-SEC-003).
- All client–server communication uses HTTPS (SRS Section 3.4).
- The institution's attendance rules apply, including a configurable minimum-attendance threshold (SAM-F-017).
- The course guidelines mandate Git and GitHub with code reviews, a CI/CD pipeline, Docker, static analysis (SonarQube or similar), security validation, and unit tests reported with code and branch coverage. Section 3.6 records the resulting toolchain.
- Development and testing run on team members' own laptops; Chrome is the primary test browser (STP Section 6).

3.2 Stakeholders and Concerns

Stakeholder	Primary concerns
Students	Seeing only their own accurate attendance; clear low-attendance warnings; a simple interface
Faculty	Fast, error-free attendance marking; ability to correct mistakes; reports for their subjects
Administrators	Easy management of students, faculty, subjects, the threshold, and access; auditability; backups
Course coordinator / evaluators	Traceability from requirements to design to tests; quality of documentation
Developers	Modularity, clear module boundaries, one well-known framework
QA / test engineers	Testable modules, seedable test data, deterministic behaviour (e.g. database unique constraints)

3.3 Component (UML) Diagram

Figure 3.1 shows the logical components of SAMS in four layers. Every request from the Web UI passes through the Authentication & Access Control component before reaching a business module. Solid arrows show calls; dashed arrows show audit-log writes.

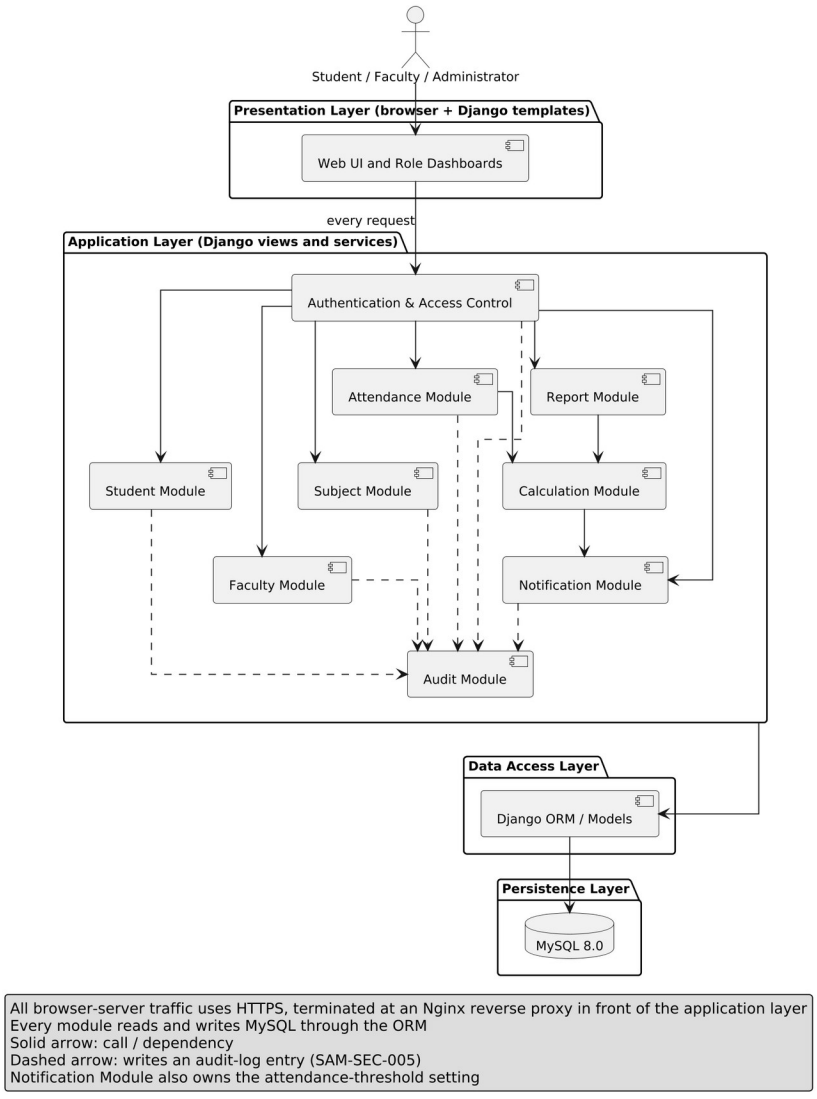


Figure 3.1 — SAMS UML component diagram (layered architecture)

3.4 Component Descriptions

Component	Layer	Responsibility	Design Spec ID(s)
Web UI / Dashboards	Presentation	Renders pages with Django templates and Bootstrap: login page, role-specific dashboards, attendance-marking sheet, report views, and user-facing error messages.	DS-UI-01
Authentication & Access Control	Application	Login/logout and sessions via Django auth; a default-deny role check on every protected view; assignment checks for faculty; binds students to their own records; locks a username and IP address after repeated failed logins; redirects unauthenticated users to login.	DS-Auth-01 to 03, DS-Security-01 to 04
Student Module	Application	Add, update, search/view, and deactivate student records (Administrator); supplies the active enrolled-student list to the Attendance Module; serves the student's own attendance history on the 'My Attendance' page, reading attendance records through the ORM.	DS-Student-01 to 03, DS-View-01
Faculty Module	Application	Add and manage faculty details; shows assigned subjects on the faculty dashboard.	DS-Faculty-01

Component	Layer	Responsibility	Design Spec ID(s)
Subject Module	Application	Create and manage subjects (a subject covers both its theory and lab classes); assign subjects, per class, to faculty; define the one-hour class time slots used when marking attendance.	DS-Subject-01, 02
Attendance Module	Application	Select class and subject, mark Present/Absent for a date and one or more time slots, store records in one transaction, reject duplicates, and let authorised users modify records.	DS-Attendance-01 to 05
Calculation Module	Application	Computes attendance percentage per student and subject from stored records.	DS-Calc-01
Notification Module	Application	Owns the attendance-threshold setting (Administrator can change it) and produces low-attendance warnings for student and faculty views.	DS-Alert-01
Report Module	Application	Builds student-wise, subject-wise, and class-wise reports as printable pages and downloadable CSV/PDF files.	DS-Report-01 to 03
Audit Module	Application	Appends audit entries (user, action, target, outcome, timestamp); read-only view for administrators; no edit or delete path. Also called by the backup and restore commands.	DS-Security-05
Django ORM / Models	Data access	Maps entities to tables, enforces constraints, and wraps writes in transactions; query design for performance and scale.	DS-Perf-01, DS-Scale-01
MySQL 8.0	Persistence	Stores all data with foreign keys, unique constraints, and transactions; target of scheduled backups.	DS-Recovery-01

Mapping to the SRS and STP module names. The SRS RTM and the STP name an Authentication Module, an Access Control Module, and a Security Module; these are built together as the Authentication & Access Control component above. The RTM's Database Module is the Django ORM with MySQL, and its generic Web/Application, System, and Application modules refer to the application as a whole (Web UI, application layer, and deployment). Two design specs describe the application as a whole rather than one component: DS-Maint-01 (the layered, per-app structure, Section 3.5) and DS-Reliability-01 (error handling, monitoring, backup, and redeploy, Sections 3.7 and 4.4). All other module names are the same as in the SRS RTM.

3.5 Chosen Architecture Pattern and Rationale

SAMS uses a layered, monolithic architecture (Presentation → Application → Data Access → Persistence), implemented with Django's MVT structure. Each functional area is a separate Django app with its own models, views, and templates, which keeps responsibilities separated and supports the maintainability goal (SAM-NF-006).

Why layered: clear separation of concerns; maps directly onto Django; easy for a four-person team to divide by module; simple to test with unit tests per module and integration tests across modules.

Why not microservices: rejected as overly complex for an institution-level class project. Separate deployments, inter-service networking, and distributed data would add operational cost without a scale requirement to justify it. A well-modularised monolith can still be split later along module lines (ADR-001).

3.6 Technology Stack and Data Stores

Application stack

Concern	Choice	Notes
Frontend	HTML, CSS, JavaScript, Bootstrap	Responsive layout for desktop, tablet, and phone (SRS Section 3.1)
Backend	Python 3, Django	Built-in authentication, sessions, CSRF protection, ORM, and templates
Database	MySQL 8.0 via the Django ORM	Foreign keys, unique constraints, transactions

Concern	Choice	Notes
Transport security	HTTPS (TLS)	All browser–server traffic; database reachable only from the application
Web server	Nginx reverse proxy in front of Django (gunicorn)	Terminates HTTPS (TLS), serves static files, and applies request rate limits
Login lockout	django-axes (Django package)	Locks a username and IP address for 15 minutes after 5 failed logins and keeps its counters in its own tables; the limits are configurable. Django has no built-in lockout
Password hashing	Django default PBKDF2 (Argon2 optional)	Never stored or logged in plain text
Reports	Django views with CSV/PDF export	Printable page and downloadable file (SAM-F-020)
Backups	Scheduled mysqldump, run through a Django management command	The command also writes the audit entry (SAM-SEC-005). The Administrator restores with a second documented command (UC: Manage Database Backup); restore verified by TC-Recovery-01

Development and delivery toolchain

Concern	Choice	How it is used and why
Version control and code review	Git, GitHub, pull requests	Every change reaches main through a pull request with at least one reviewer (guidelines: code reviews mandatory; per-member commits are checked)
Backlog, sprints, defects	GitHub Issues and GitHub Projects board	Guidelines accept 'GitHub (Jira or similar)' and require agile development. User stories and the sprint backlog live as issues on the Projects board, next to the pull requests and defects; the STP already uses GitHub Issues
CI/CD	GitHub Actions workflow (a YAML file in the repository)	The guidelines require a CI/CD pipeline. It runs on every pull request and every push to main, on GitHub-hosted runners, so there is no server for the team to keep running. Stages: checkout, install, unit tests with coverage (MySQL as a service container), Bandit scan, SonarCloud scan, build the Docker image; on merge to main, publish the image to GitHub Container Registry
Containers	Docker, docker-compose	One container each for Nginx, the Django application (gunicorn), and MySQL; every member runs the same environment. Deployment means pulling the published image and running docker-compose up on the demo machine (staging)
Static analysis and security scan	SonarCloud (hosted SonarQube); Bandit	The guidelines list 'SonarQube or similar'. SonarCloud is free for public repositories and plugs into the GitHub Actions workflow, so no SonarQube server is needed. Bandit is a lightweight Python security scanner that adds automated security validation
Testing and coverage	pytest-django / Django TestCase; coverage.py with branch coverage	Matches the STP; produces the unit test report with code and branch coverage that the guidelines require

Toolchain decision. The guidelines make a CI/CD pipeline, code reviews, and security validation mandatory, and list Jenkins, Docker, and SonarQube (or similar) as tools. This project uses GitHub Actions for the pipeline: the code, reviews, issues, and pipeline stay in one place, the workflow runs on GitHub-hosted runners with nothing for a four-person team to host, and the STP and the project README already name GitHub Actions, so all three documents agree. SonarCloud fills the static-analysis slot, and Docker is used as listed. The pipeline stages are tool-neutral and could be moved to another CI tool if required (ADR-008). Continuous deployment targets a staging environment on the team's own machine; production hosting is outside the scope of this project.

Data stores. A single MySQL database holds all persistent data: user accounts, students, faculty, classes, subjects, subject assignments, attendance records, audit logs, and system settings (Section 4.1.1). No other data store, cache, or external system integration is needed for the current scope.

3.7 Risks and Mitigations

Risk	Impact	Mitigation
Database or network outage during academic hours	Faculty cannot mark attendance; availability target missed	Clear error page instead of silent failure; scheduled backups and a documented restore; Docker setup lets the stack be redeployed quickly
Double submit or two faculty marking the same session	Duplicate or conflicting records, wrong percentages	Database unique constraint on (student, subject, date, time slot) plus atomic transactions; duplicate-rejection message (SAM-F-013)
A theory class and a lab of one subject on the same day	If attendance were kept per day only, the second class would be rejected as a duplicate	Attendance is recorded per one-hour time slot; the unique key is (student, subject, date, time slot), so both classes are stored (ADR-004). SAM-F-012 and SAM-F-013 are refined accordingly; the SRS wording is to be updated
Report generation slow on large data	3-second target missed	Indexes on (student, subject, date) and (subject, date); aggregation in the database; pagination; measured by TC-Perf-01 once a build exists
A new view is added without a role check	Privilege escalation or data leak	Default-deny access mixin on all views; automated tests that call every URL as every role
Hard-deleting a student orphans history	Lost attendance history	Deactivation instead of deletion (ADR-003)
Action succeeds but its audit entry is lost	Unaccountable change (SAM-SEC-005)	Audit entry written in the same transaction as the change (ADR-007)
Uneven familiarity with Django testing and GitHub Actions	Slow or inconsistent tests and pipeline	Internal walkthrough before automation begins (STP Section 10)
Pipeline or SonarCloud setup takes longer than planned	CI/CD and security validation delayed	Set the workflow up in the first sprint: start with tests and coverage, then add Bandit, SonarCloud, and the image build one at a time (ADR-008)

3.8 Traceability to Requirements

Every requirement in the SRS RTM is assigned to a design component below. The Design Spec IDs are the ones already cited in SRS Section 8, and the Test Case IDs are the ones used by the STP. This includes SAM-F-008, SAM-F-009, and SAM-F-014, which are designed here even though the STP's use-case table (STP Section 13) does not list them (see Section 4.6). SAM-F-012 and SAM-F-013 are implemented at time-slot granularity (ADR-004), so their SRS wording is to be updated.

Req ID	Requirement	Component / Design Spec	Test Case
SAM-F-001	User login	Authentication & Access Control / DS-Auth-01	TC-Auth-01
SAM-F-002	Invalid login handling	Authentication & Access Control / DS-Auth-02	TC-Auth-02
SAM-F-003	Role-based access	Authentication & Access Control / DS-Auth-03	TC-Auth-03
SAM-F-004	Add student	Student Module / DS-Student-01	TC-Student-01
SAM-F-005	Update student	Student Module / DS-Student-02	TC-Student-02
SAM-F-006	Search student	Student Module / DS-Student-03	TC-Student-03
SAM-F-007	Manage faculty	Faculty Module / DS-Faculty-01	TC-Faculty-01
SAM-F-008	Manage subjects	Subject Module / DS-Subject-01	TC-Subject-01
SAM-F-009	Assign subject to faculty	Subject Module / DS-Subject-02	TC-Subject-02
SAM-F-010	Select class and subject	Attendance Module / DS-Attendance-01	TC-Att-01
SAM-F-011	Mark attendance	Attendance Module / DS-Attendance-02	TC-Att-02
SAM-F-012	Store attendance record	Attendance Module, Models (date and time slot stored) / DS-Attendance-03	TC-Att-03
SAM-F-013	Prevent duplicate attendance	Unique constraint (student, subject, date, time slot) / DS-Attendance-04	TC-Att-04

Req ID	Requirement	Component / Design Spec	Test Case
SAM-F-014	Modify attendance	Attendance Module / DS-Attendance-05	TC-Att-05
SAM-F-015	Calculate percentage	Calculation Module / DS-Calc-01	TC-Calc-01
SAM-F-016	View attendance history	Student Module (own-history view) / DS-View-01	TC-View-01
SAM-F-017	Low-attendance warning	Notification Module / DS-Alert-01	TC-Alert-01
SAM-F-018	Student-wise report	Report Module / DS-Report-01	TC-Report-01
SAM-F-019	Subject / class report	Report Module / DS-Report-02	TC-Report-02
SAM-F-020	Download / print report	Report Module / DS-Report-03	TC-Report-03
SAM-NF-001	Response time ≤ 3 s	Indexing, database aggregation / DS-Perf-01	TC-Perf-01
SAM-NF-002	99% availability	Error handling, monitoring, Docker redeploy / DS-Reliability-01	TC-Reliability-01
SAM-NF-003	User-friendly interface	Web UI (Section 4.5) / DS-UI-01	TC-UX-01
SAM-NF-004	Data protection	Security architecture (Section 3.9) / DS-Security-01	TC-Sec-01
SAM-NF-005	Backup and recovery	Scheduled mysqldump and restore / DS-Recovery-01	TC-Recovery-01
SAM-NF-006	Maintainability	Layered, per-app modules / DS-Maint-01	TC-Maint-01
SAM-NF-007	Multiple simultaneous users	Stateless app, indexed queries / DS-Scale-01	TC-Load-01
SAM-SEC-001	Authentication required	Access Control (default-deny) / DS-Security-01	TC-SEC-01
SAM-SEC-002	Role-based authorization	Access Control / DS-Security-02	TC-SEC-02
SAM-SEC-003	Secure password storage	Django PBKDF2 hashing / DS-Security-03	TC-SEC-03
SAM-SEC-004	Protect student attendance data	Ownership checks in Access Control / DS-Security-04	TC-SEC-04
SAM-SEC-005	Audit important actions	Audit Module / DS-Security-05	TC-SEC-05

3.9 Security Architecture

Security controls are layered. Transport is protected by HTTPS; every request passes through the Authentication & Access Control component before reaching any business module (default-deny); object-level checks bind students to their own records; passwords are hashed by Django; and sensitive actions, including denied attempts, are written to an append-only audit log in the same transaction as the action. Security validation is also part of the build: the GitHub Actions workflow runs the SonarCloud scan, a Bandit security scan, and the per-role access tests on every push and pull request.

Threat modelling (STRIDE)

Threat	Example in SAMS	Mitigation
Spoofing	Logging in as another user by guessing or reusing credentials	Authentication required (SAM-SEC-001); PBKDF2-hashed passwords; secure, HTTP-only session cookies; a failed-login lockout (django-axes: 5 failures, 15 minutes); one generic invalid-login message
Tampering	A student or unassigned faculty edits attendance; altering hidden form fields	Role and assignment checks on every write; server-side validation; CSRF tokens; unique constraint; every modification audited with who and when
Repudiation	A user denies changing attendance or an account	Audit log with user, action, target, outcome, and timestamp; append-only and not editable through the UI (SAM-SEC-005)
Information disclosure	A student changes an ID in the URL to see another student's records; data read in transit	Student identity taken from the session, never from the request (SAM-SEC-004); HTTPS; no sensitive data in logs or error pages; database reachable only from the application
Denial of service	Flooding login or report endpoints	Pagination and bounded report date ranges; login lockout; Nginx rate limiting; monitoring
Elevation of privilege	Faculty or a student opens admin pages by typing the URL	RBAC enforced server-side on every view (SAM-SEC-002); hidden menu items are not relied on; automated per-role URL tests

4. Design

4.1 Design Overview

SAMS is built as a set of Django apps that mirror the modules in the SRS RTM: auth, students, faculty, subjects, attendance, reports, and audit. Each app has the same internal structure: models (data), services (business rules such as percentage calculation and duplicate handling), views (request handling and permission checks), and templates (presentation). Keeping business rules in services means they can be unit-tested without a browser.

Design rules:

- Every view is protected by a role check unless explicitly marked public (the login page only).
- All attendance writes happen inside one database transaction: every row is saved, or none.
- Business logic lives in service functions, never in templates.
- Administrative and security-relevant actions always call the Audit Module, inside the same transaction as the change.

4.1.1 Data Model

Figure 4.1 shows the persistent entities. The college takes attendance by time, in one-hour slots. A subject has several classes, some of them labs, and a lab and a theory class of the same subject can fall on the same day. Each hour is marked against a TimeSlot (a defined one-hour period such as 09:00–10:00), and Attendance is unique on (student, subject, date, time slot). A one-hour theory class is one entry per student; a two-hour lab occupies two consecutive slots and is stored as two entries, so it counts as two classes. Lab and theory classes of a subject are one subject and count together in one percentage. SAM-F-012 and SAM-F-013 in the SRS name only student, subject, and date; adding the time slot refines them, and the SRS wording is to be updated (ADR-004).

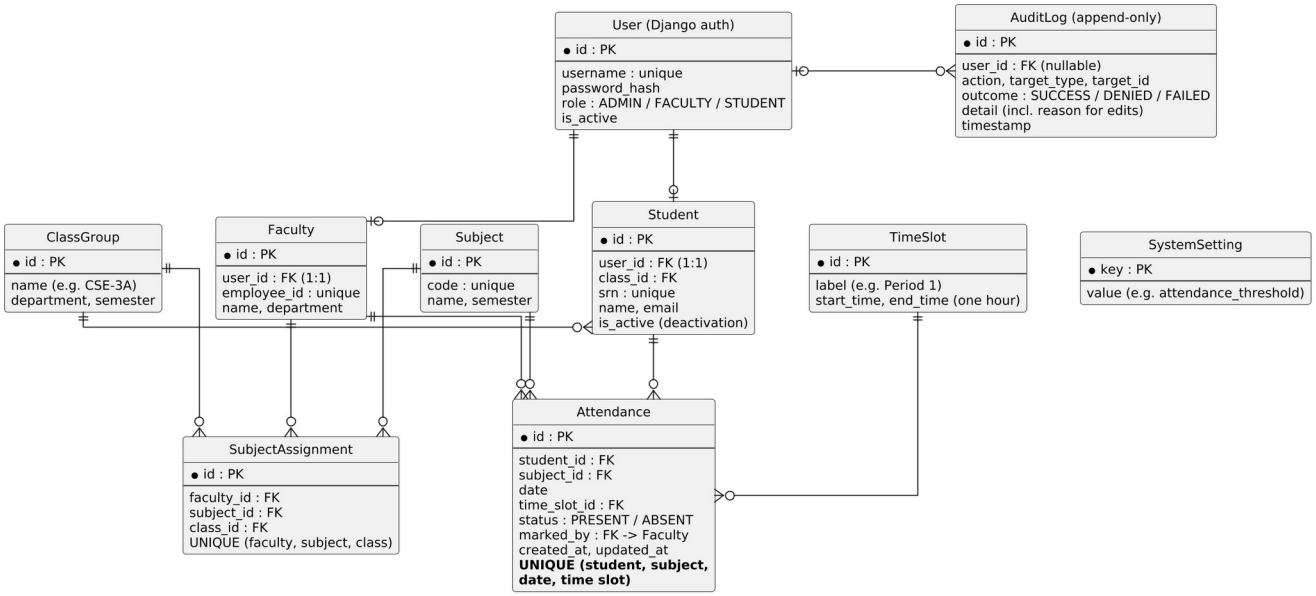


Figure 4.1 — SAMS entity-relationship diagram

Attendance percentage. For a student and subject: percentage = (classes marked PRESENT ÷ total classes recorded) × 100, rounded to two decimals. Each one-hour time slot is one class, so a one-hour theory class counts as 1 and a two-hour lab counts as 2; all of a subject's classes, theory and lab, count together in this one figure. If no classes have been recorded yet, the percentage is shown as 'No data' rather than 0%, and no warning is raised.

Low-attendance rule. A warning is produced when a subject's percentage is below the threshold stored in SystemSetting. The SRS requires the threshold to be configurable but does not fix a value, so the design ships with a default of 75%, which the Administrator can change.

Login lockout. The lockout package keeps its failed-login counters in its own tables, outside the SAMS domain model in Figure 4.1. The limits (5 failures, 15 minutes) are configurable defaults, since the SRS does not fix them.

4.1.2 Division of Work

The work is divided by the roles in STP Section 9: two Developers (Anmol Vyas and Anupam Rajesh Tiwari), a QA Lead (Anupriya), and a Test Engineer (Anusha G). The guidelines ask each member to complete one full functionality and to contribute code (git commits are checked), so every member implements a functionality. The two Developers carry the largest share of the build, the QA Lead and Test Engineer carry the largest share of the quality work, and every member's total is the same. Apart from login and access control, anyone who builds a module is not the one who tests it, which also keeps the STP's use-case test ownership unchanged.

Table 1: Who does what

Member and role	Implementation (one full functionality plus supporting items)	Testing and role duties	Points (build + test)
Anmol Vyas Developer	Full functionality: attendance recording and correction (Attendance Module; SAM-F-010 to SAM-F-014). Supporting: data model, migrations, and seed data; Docker, docker-compose, and Nginx environment; backup and restore commands.	Test owner for Generate Reports (UC-06, as in the STP) and for subject management (SAM-F-008, SAM-F-009). Fixes defects in his modules.	9 + 3 = 12
Anupam Rajesh Tiwari Developer	Full functionality: reports with print and CSV/PDF export (Report Module; SAM-F-018 to SAM-F-020). Supporting: login and role-based redirect (SAM-F-001 to SAM-F-003); access control (SAM-SEC-001, 002, 004); password hashing and login lockout (SAM-SEC-003).	Test owner for Mark Attendance and Select Class and Subject (UC-01, UC-02, as in the STP) and for attendance modification (SAM-F-014). Fixes defects in his modules.	9 + 3 = 12
Anupriya QA Lead	Full functionality: monitoring and alerts (Calculation Module, Notification Module with the threshold setting, student attendance history; SAM-F-015 to SAM-F-017). Supporting: subjects, assignments, and time slots (SAM-F-008, SAM-F-009); the GitHub Actions pipeline with coverage, Bandit, and SonarCloud.	Sets the quality gates (coverage target, SonarCloud quality gate) and the pull-request review rules, triages defects, and coordinates the plan. Test owner for Manage User Access and Log Administrative Action (UC-04, UC-05, as in the STP).	8 + 4 = 12
Anusha G Test Engineer	Full functionality: student and faculty management with deactivation (SAM-F-004 to SAM-F-007). Supporting: Audit Module (SAM-SEC-005); base templates and role dashboards (SAM-NF-003).	Test owner for View Attendance Percentage and View Attendance History (UC-03, UC-07, as in the STP). Defines the test data sets, keeps the RTM status column current, and compiles the metrics and test summary report.	7 + 5 = 12

Table 2: Implementation work items

#	Work item	Requirements	Owner	Points
1	Attendance marking: marking sheet, one or more time slots, one transaction, duplicate rejection	SAM-F-010 to 013	Anmol Vyas	3
2	Attendance modification with a reason	SAM-F-014	Anmol Vyas	1
3	Data model, migrations, and seed data	Foundation (Figure 4.1)	Anmol Vyas	2
4	Docker, docker-compose, and Nginx environment	Deployment (Section 3.6)	Anmol Vyas	2

#	Work item	Requirements	Owner	Points
5	Backup and restore commands	SAM-NF-005	Anmol Vyas	1
6	Report views: student-wise, subject-wise, class-wise	SAM-F-018, 019	Anupam Rajesh Tiwari	2
7	Report export and print (CSV, PDF)	SAM-F-020	Anupam Rajesh Tiwari	2
8	Login and role-based redirect	SAM-F-001 to 003	Anupam Rajesh Tiwari	2
9	Access control: default-deny, assignment and ownership checks	SAM-SEC-001, 002, 004	Anupam Rajesh Tiwari	2
10	Password hashing and login lockout	SAM-SEC-003	Anupam Rajesh Tiwari	1
11	Attendance percentage calculation	SAM-F-015	Anupriya	1
12	Threshold setting and low-attendance flags	SAM-F-017	Anupriya	2
13	Student attendance history	SAM-F-016	Anupriya	1
14	Subjects, assignments, and time slots	SAM-F-008, 009	Anupriya	2
15	GitHub Actions pipeline: tests and coverage, Bandit, SonarCloud, image build	Course guidelines	Anupriya	2
16	Student management: add, update, search, deactivate	SAM-F-004 to 006	Anusha G	2
17	Faculty management	SAM-F-007	Anusha G	1
18	Audit Module	SAM-SEC-005	Anusha G	2
19	Base templates and role dashboards	SAM-NF-003	Anusha G	2

Points are relative effort estimates (1 small, 2 medium, 3 large) used to keep the split even; the Project Plan will replace them with a full work breakdown and effort estimates. Build points: Anmol 9, Anupam 9, Anupriya 8, Anusha 7. Test and role points: owning the tests of a use-case group is 2; owning the tests of one extra requirement (SAM-F-014, or SAM-F-008 and SAM-F-009) is 1; quality gates and review rules are 2; test data, RTM upkeep, and the metrics and summary report are 1 each.

Table 3: Who builds and who tests

Area (requirements)	Implemented by	Tested by
Mark attendance, select class and subject, modify attendance (UC-01, UC-02; SAM-F-010 to 014)	Anmol Vyas	Anupam Rajesh Tiwari
Reports (UC-06; SAM-F-018 to 020)	Anupam Rajesh Tiwari	Anmol Vyas
Monitoring, alerts, and history (UC-03, UC-07; SAM-F-015 to 017)	Anupriya	Anusha G
Student and faculty management and audit (UC-04, UC-05; SAM-F-004 to 007, SAM-SEC-005)	Anusha G	Anupriya
Subjects and assignments (SAM-F-008, 009)	Anupriya	Anmol Vyas
Login, access control, password hashing, and lockout (SAM-F-001 to 003, SAM-SEC-001 to 004)	Anupam Rajesh Tiwari	Shared preconditions (STP); SAM-SEC-002 checked under UC-04 and SAM-SEC-004 under UC-07

Shared by all four members:

- Every member commits code to the repository, and every pull request is reviewed by a member who did not write it.
- The non-functional test cases (TC-Perf-01, TC-Load-01, TC-UX-01, TC-Maint-01, TC-Reliability-01, TC-Recovery-01) are shared: each member takes the ones for areas they did not implement, assigned in the Project Plan.
- If a member is unavailable, items move to the others in a way that keeps the points even. SAM-F-008, SAM-F-009, and SAM-F-014 are not yet in the STP use-case table; the STP update adds them with the test owners above.

4.2 UML Sequence Diagrams

Four flows are shown: login (SD-01), marking attendance (SD-02), viewing attendance percentage with the low-attendance alert (SD-03), and managing user access with audit logging (SD-04). Alternative outcomes are shown as alt fragments.

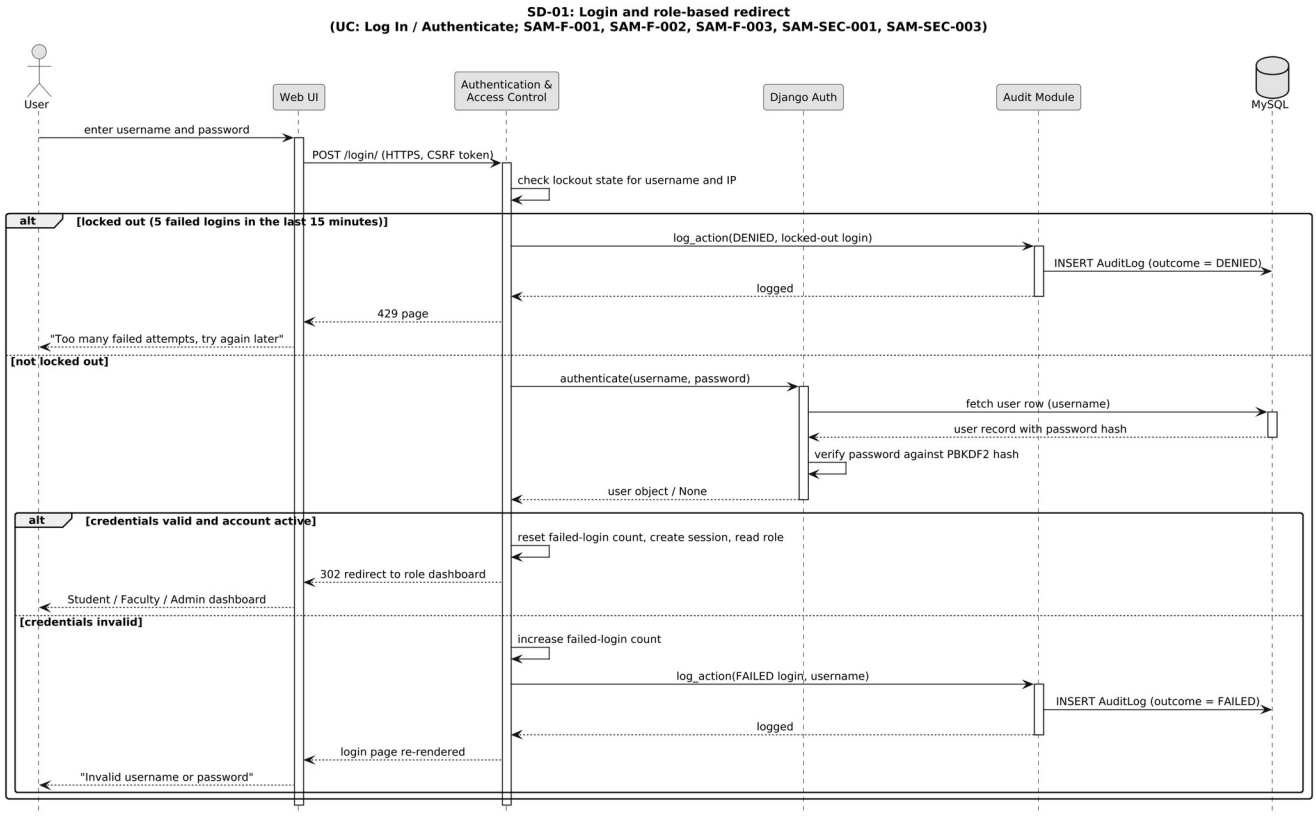


Figure 4.2 — SD-01: Login and role-based redirect

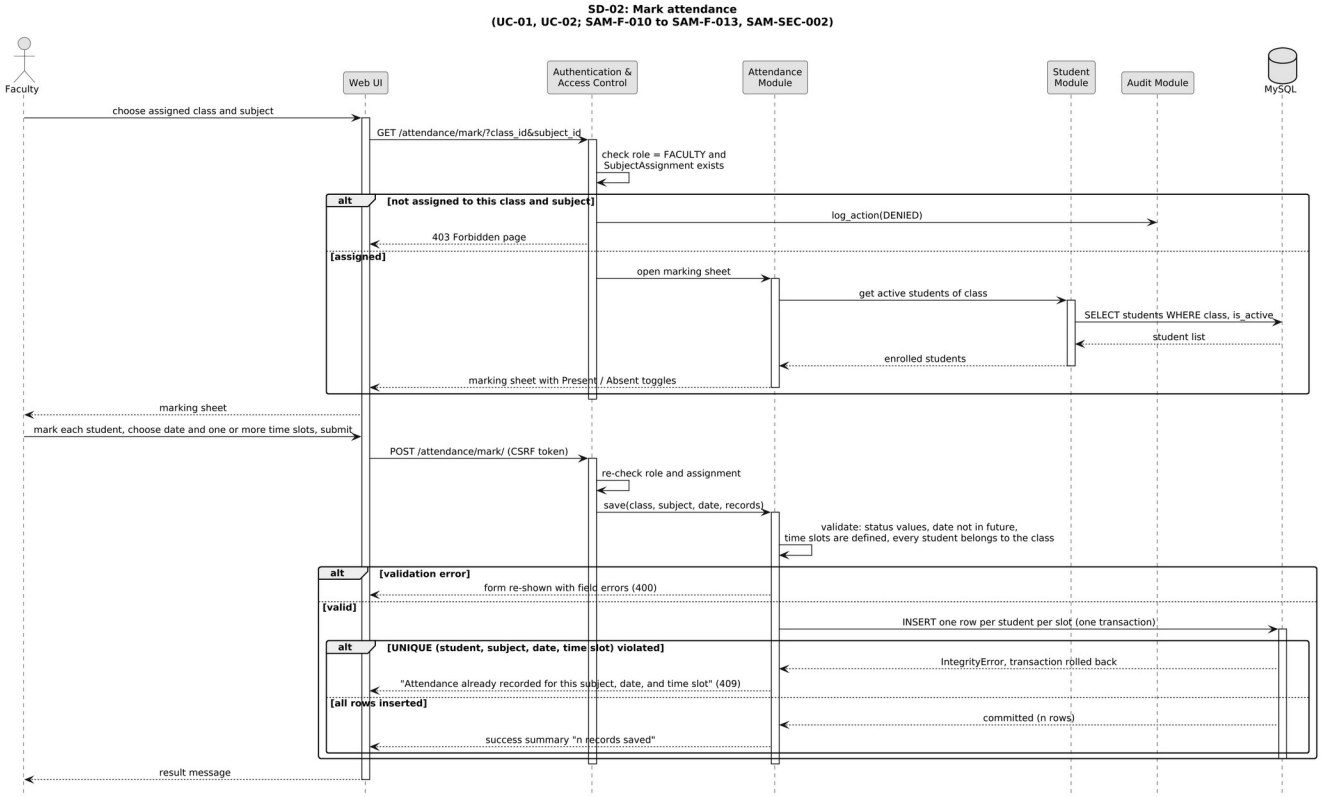


Figure 4.3 — SD-02: Mark attendance

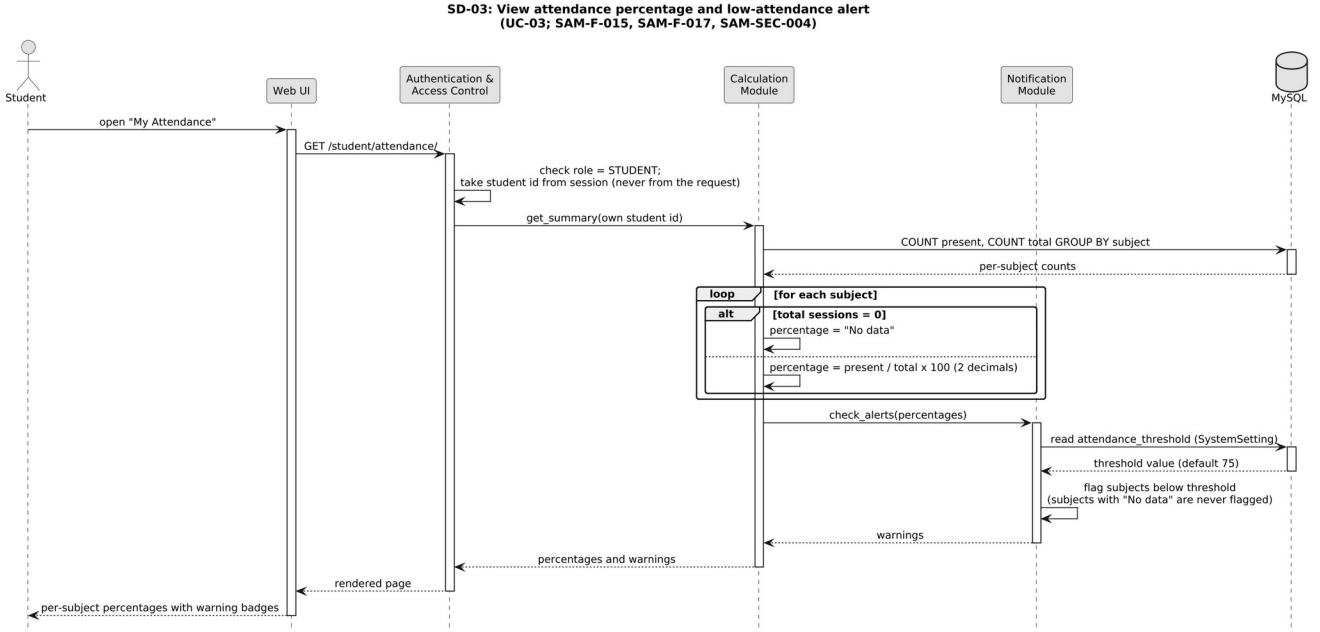


Figure 4.4 — SD-03: View attendance percentage and low-attendance alert

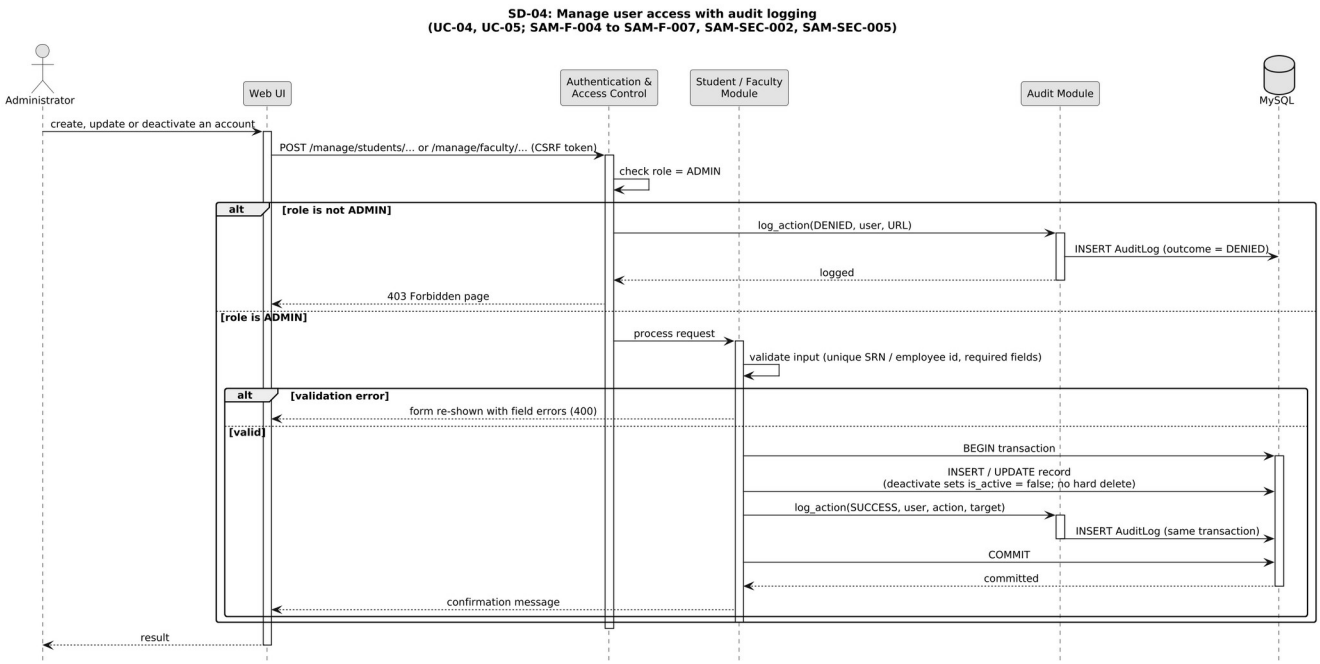


Figure 4.5 — SD-04: Manage user access with audit logging

4.3 API Design

SAMS is a server-rendered web application (ADR-006), so its interfaces are HTTP endpoints handled by Django views. Requests are HTML form submissions carrying a CSRF token; responses are rendered pages or redirects, with the HTTP status codes listed below. All endpoints require HTTPS and an authenticated session unless marked Public. The Authentication and Attendance components are defined in full; the remaining endpoints follow in summary form. An ABI definition does not apply: SAMS is interpreted Python with no compiled binary interfaces between components, so its stable interfaces are the HTTP endpoints below and the database schema (Section 4.1.1).

4.3.1 Authentication Component

Endpoint	Method	Access	Request (form fields)	Response	Errors
/login/	POST	Public	username, password	302 redirect to the role dashboard; session cookie set	200, login page re-shown with 'Invalid username or password' (SAM-F-002); attempt audited as FAILED. 429 'Too many failed attempts, try again later' once the lockout limit is reached, audited as DENIED
/logout/	POST	Authenticated	—	302 redirect to /login/; session destroyed	—
Any protected URL	GET / POST	Authenticated	—	Requested page	302 to /login/ if not logged in (SAM-SEC-001); 403 if the role is not permitted (SAM-SEC-002), audited as DENIED

4.3.2 Attendance Component

Endpoint	Method	Access	Request (form fields)	Response	Errors
/faculty/assignments/	GET	Faculty	—	Page listing the faculty member's assigned class and subject pairs	403 for other roles
/attendance/mark/	GET	Faculty (assigned)	query: class_id, subject_id	Marking sheet with the class's active students	403 if not assigned; 404 unknown class or subject

Endpoint	Method	Access	Request (form fields)	Response	Errors
/attendance/mark/	POST	Faculty (assigned)	class_id, subject_id, date, one or more time_slot_ids, one status per student	Page with 'n records saved' (one per student per slot)	400 invalid status, future date, undefined time slot, or student not in class; 409 duplicate (student, subject, date, time slot), nothing saved; 403
/attendance/<id>/edit/	POST	Faculty (assigned) / Admin	status, reason	Updated record page; audit entry written, with the reason in its detail field	403 if not authorised; 404; 400 invalid status
/student/attendance/	GET	Student (own data)	optional: subject_id	Per-subject percentages, warnings, and history (newest first)	403 for other roles; the student id is taken from the session, never from the request

4.3.3 Other Endpoints (summary)

Area	Endpoints	Access	Notes
Students	/manage/students/ (list, search), /manage/students/add/, /manage/students/<id>/edit/, /manage/students/<id>/deactivate/	Admin (search: Faculty, read-only)	Deactivate instead of delete (ADR-003); every write is audited
Faculty, subjects, and time slots	/manage/faculty/, /manage/subjects/, /manage/assignments/, /manage/timeslots/	Admin	Assignments drive the faculty dashboard (SAM-F-009); time slots are the periods offered when marking attendance; every write is audited
Reports	/reports/student/, /reports/subject/, /reports/class/ with from, to, and format = html, csv, or pdf	Faculty (assigned subjects) / Admin (all)	The format parameter gives print and download (SAM-F-020). Faculty view a student's history for their classes through the student-wise report (UC-07), and the student-wise and subject-wise reports flag students below the threshold, which gives faculty the low-attendance warning (SAM-F-017)
Settings and audit	/manage/settings/threshold/, /manage/audit/	Admin	Threshold changes are audited; the audit view is read-only
Backup and restore	No web endpoint	Administrator (server access)	Two Django management commands, backup_db and restore_db, wrap mysqldump and the restore (UC: Manage Database Backup, SAM-NF-005). A scheduler runs backup_db and the Administrator runs restore_db. Each run writes an AuditLog entry (SAM-SEC-005; user left empty for scheduled runs). Kept off the web interface because a backup holds all personal data

Standard responses. 302 to login when not authenticated; 403 when authenticated but not permitted; 429 too many failed logins (temporary lockout); 404 not found; 400 validation error (form re-shown with field messages); 409 duplicate attendance; 500 generic error page. Messages never expose stack traces, SQL, or other users' data.

4.4 Error Handling, Logging, and Monitoring

Error handling

- Standard, user-friendly messages shown inline on forms; custom 403, 404, and 500 pages; a clear message when the database or network is unavailable rather than failing silently (SRS Section 3.4).
- Server-side validation of every input (status values, dates not in the future, defined time slots, student belongs to the selected class); client-side checks are never relied on alone.

- A duplicate attendance record is caught by the database unique constraint and reported as a duplicate, not as a server error.
- Multi-row attendance saves use one transaction, so a failure part-way leaves no partial data.

Logging

- Application log (rotating files): errors, warnings, and failed requests with a timestamp and request id.
- Audit log (append-only database table): user, action, target, outcome (SUCCESS, DENIED, or FAILED), and timestamp for administrative actions, attendance modifications, failed logins, and denied access attempts. The entry is written in the same transaction as the change it records.
- No passwords, session tokens, or full personal records are written to any log.

Monitoring

- Availability (SAM-NF-002): an uptime check on the login page during academic hours.
- Response time (SAM-NF-001): requests slower than 3 seconds are logged.
- Failure indicators: rate of failed logins, 403s, and 500s; backup job success or failure; database disk usage; GitHub Actions workflow and SonarCloud quality-gate status.

4.5 UX Design

- One login page for all roles, followed by a role-specific dashboard (Student: percentages and alerts; Faculty: assigned subjects and quick marking; Administrator: management menu).
- Attendance-marking sheet: pick the date and one or more time slots (a two-hour lab uses two consecutive slots), then one row per student with a Present/Absent toggle, a 'mark all present' shortcut, and a clear save confirmation, so a class can be marked in under a minute. The same marks are saved for every selected slot, and a single hour can be corrected afterwards through the edit endpoint.
- Low-attendance subjects carry a warning badge with text, not colour alone, for accessibility.
- Responsive Bootstrap layout on desktop, tablet, and phone; consistent navigation and primary buttons; readable font sizes and sufficient contrast.
- Reports in clear tables with filters (type, subject or class, date range) and visible Print and Download buttons.
- Inline validation messages; deactivation and other sensitive actions ask for confirmation.

4.6 Open Issues and Next Steps

The five open items raised in STP Section 13.1 are resolved as follows. Four of the resolutions stay within the SRS v1.0 requirements as written; the time-slot decision refines SAM-F-012 and SAM-F-013, so those two SRS entries are to be reworded.

Open item (STP 13.1)	Design decision	Status
Multiple sessions of the same subject on one day	The college takes attendance in one-hour slots, so each hour is recorded against a time slot and Attendance is unique on (student, subject, date, time slot). A lab and a theory class of one subject on the same day are separate entries, and a two-hour lab is two entries (two classes). Lab and theory still count together in one percentage. This is the slot field the STP proposed; SAM-F-012 and SAM-F-013 wording is to be updated in the SRS (ADR-004).	Done: built in Sprint 1; SRS v1.1 reworded
Student deactivation vs deletion	Deactivate (is_active = false), as SRS UC-04 already describes; no hard delete through the UI; history preserved; deactivated students are left off new marking sheets (ADR-003).	Done (built as decided)
Audit logging of denied or failed actions	Logged with outcome DENIED or FAILED. This goes beyond the minimum in SAM-SEC-005 and closes the repudiation threat (ADR-005).	Done (built as decided)

Open item (STP 13.1)	Design decision	Status
Report generation performance	Indexes on (student, subject, date) and (subject, date); aggregation in the database; pagination. Measured with TC-Perf-01 on a realistic seeded dataset.	Done: TC-Perf-01 passes (every page under 3 s); two more indexes and paging added
Default ordering of attendance history	Date order, newest first, which keeps the 'chronological' display in SRS UC-07.	Done (built as decided)

Next steps

- In the first sprint: set up the Django project skeleton, Docker and docker-compose files, the GitHub Actions workflow, and the SonarCloud scan; add a seed script for test data.
- STP test-case update (Deliverables Part-1, item 4): add SAM-F-008, SAM-F-009, and SAM-F-014 to the use-case coverage in STP Sections 3 and 13 (with the test owners in Section 4.1.2), and add Docker, SonarCloud, Bandit, and coverage.py to the tooling in STP Section 6. The STP's CI tool (GitHub Actions) already matches.
- Update the SRS wording for the time-slot key (SAM-F-012, SAM-F-013, and the UC-01 description) and the matching STP test steps (TC-Att-03, TC-Att-04); both are small refinements, not new requirements.
- Update the project README to match this document: the resolved design questions, the added tooling (Docker, SonarCloud, Bandit), and the repository structure.
- Confirm the institutional minimum-attendance percentage with the course coordinator; 75% is only the configurable default.
- Possible future enhancements: email or SMS alerts, parent notifications, QR-code or biometric attendance capture, analytics dashboards.

Status in version 1.1: the first-sprint set-up, the STP and SRS updates and the README update are done (STP v1.1, SRS v1.1). Still open: the institutional minimum percentage (75% stays the configurable default) and the future enhancements.

4.7 Architecture Decision Records

ID	Decision	Alternatives considered	Rationale / consequence
ADR-001	Layered monolith	Microservices	Right-sized for a class project and a four-person team; simpler deployment and testing. Can be split later along module lines.
ADR-002	Django with MySQL (fixed in SRS v1.0)	Flask or Spring Boot (the other pre-approved options); PostgreSQL	Django provides authentication, sessions, CSRF protection, ORM, and templates out of the box, which reduces security risk and effort.
ADR-003	Deactivate students and faculty instead of deleting them	Hard delete; cascade delete	Preserves attendance history and audit integrity. Every list must filter on is_active.
ADR-004	Record attendance per one-hour time slot: one record per (student, subject, date, time slot); lab and theory classes of a subject count together in one percentage	One record per (student, subject, date), as worded in SRS v1.0; one record per session with a duration or weight field; separate Subject records for theory and lab	The college takes attendance by time in one-hour slots, and a lab and a theory class of one subject can fall on the same day; without the slot the second class would be rejected as a duplicate. A two-hour lab is marked on two consecutive slots in one submission and counts as two classes, while a one-hour theory class counts as one, which matches college practice with no extra weight field. Per-slot records also let a late arrival or early exit be corrected for one hour only (SAM-F-014). Time slots are defined centrally (TimeSlot) rather than typed freehand. Refines SAM-F-012 and SAM-F-013, so the SRS wording is to be updated.
ADR-005	Audit denied and failed actions as well as successful ones	Log successful actions only	Security-relevant events become visible; the log grows faster, so a retention period is needed.
ADR-006	Server-rendered pages (Django templates)	Separate single-page app with a REST API	Fewer moving parts and one backend language; a REST API can be added later if needed.

ID	Decision	Alternatives considered	Rationale / consequence
ADR-007	Audit entry written in the same transaction as the change	Write the audit entry after the change commits	An action can never succeed without its audit record.
ADR-008	GitHub Actions for CI/CD; Docker Compose for packaging and staging; SonarCloud and Bandit for analysis; GitHub Issues and Projects for backlog and defects	Jenkins (named in the guidelines); self-hosted SonarQube; Jira	The guidelines require a CI/CD pipeline and list Jenkins among the tools. Hosted runners mean no server to maintain or keep switched on, SonarCloud avoids hosting SonarQube, and one platform holds code, reviews, pipeline, and issues. The STP and README already name GitHub Actions. The stages are tool-neutral, so they could move to a Jenkinsfile if required.

5. Appendices

5.1 Glossary

SAMS, SRS, STP, RTM, SAD, ADR, MVT, ORM, RBAC, STRIDE, TLS, HTTPS, CSRF, CI/CD: see Section 1.4. PBKDF2: Password-Based Key Derivation Function 2, Django's default password hasher. UC: Use Case, as defined in SRS Section 7. SRN: Student Registration Number.

5.2 References

- Software Requirements Specification, Student Attendance Management System, v1.0, Team 3.
- Software Test Plan, Student Attendance Management System, v1.0, Team 3.
- Project repository and README, https://github.com/anu-9t/Student-Attendance-Management-System
- SE Mini-Project Guidelines and Mini-Project Deliverables Part-1, PES University, Dept. of CSE.
- Department Software Architecture and Design Specification template (ATM System sample).
- ISO/IEC/IEEE 42010, Systems and software engineering: Architecture description.
- OWASP Top 10 and OWASP Application Security Verification Standard.
- Django Software Foundation, "Django Documentation", https://docs.djangoproject.com/
- Oracle Corporation, "MySQL 8.0 Reference Manual", https://dev.mysql.com/doc/

5.3 Tools

- PlantUML: all component, entity-relationship, and sequence diagrams in this document.
- Swagger/OpenAPI is not used: SAMS exposes server-rendered pages rather than a REST API (ADR-006), so the endpoint tables in Section 4.3 are the interface definition.
- Build, test, and delivery tools (Git and GitHub, GitHub Actions, Docker, SonarCloud, Bandit, pytest-django, coverage.py, Chrome) are listed in Section 3.6.

6. Changes in Version 1.1

SAMS was built in two one-week sprints (19 to 23 and 26 to 30 October 2026). The architecture, the modules, the data model and the decisions in Section 4.7 were followed. This section records where the implementation differs in detail from version 1.0, and why. The same list, for all three documents, is in docs/DEVIATIONS.md in the repository.

6.1 Interfaces as Built

The endpoints keep the access rules of Section 4.3, but their paths follow the module layout of Section 3.5 (each module includes its own URLs), so the /manage/ prefix was dropped.

Area	Endpoints	Access	Notes
Login and dashboards	/login/, /logout/ (POST), / (sends each role to its dashboard), /dashboard/admin/, /dashboard/faculty/, /dashboard/student/	Public (login); then per role	Wrong login: page re-shown with 'Wrong username or password', audited FAILED; lockout answers 429
Attendance	/attendance/select/ (was /faculty/assignments/), /attendance/mark/<class>/<subject>/, /attendance/modify/<class>/<subject>/ (was /attendance/<id>/edit/)	Faculty (assigned)	Modify works on one date and slot at a time, so one reason covers a whole correction; every changed record is audited
Student views	/percentage/ and /history/ (was /student/attendance/), /history/student/<id>/	Student (own); faculty (their classes); admin	Student id always comes from the session; other ids are 403 and audited
Management	/students/, /faculty/, /subjects/, /subjects/time-slots/, /subjects/assignments/ (add, edit, deactivate)	Admin	Deactivate instead of delete (ADR-003); every write audited
Reports	/reports/, /reports/student/<id>/, /reports/subject/<class>/<subject>/, /reports/class/<class>/; date_from, date_to, format=csv or pdf, page	Faculty (assigned) / Admin (all)	50 rows per page on screen; downloads have every row; percentages below the threshold in red
Settings, audit, health	/notifications/threshold/, /audit/ (search, outcome filter), /health/	Admin; health is public	/health/ also queries the database and answers 503 when it can't

Validation errors and duplicate attendance re-show the form with a message (HTTP 200) rather than 400 or 409, which is how Django forms report errors; nothing is saved in either case.

6.2 Other Differences from Version 1.0

Area	Version 1.0	As built, and why
Backups (SAM-NF-005)	Scheduled mysqldump wrapped in backup_db	backup_db writes Django's JSON dump, compressed; restore_db loads it. Portable between MySQL and SQLite, so TC-Recovery-01 runs in CI. Nightly runs are a host cron job (DEVELOPING.md)
Modifying attendance (SAM-F-014)	Per record; faculty or admin	Per date and slot, by the assigned faculty member only, reason required. The admin has no attendance pages, which keeps who changed what clear
Low-attendance warnings for faculty (SAM-F-017)	Through the student-wise and subject-wise reports	A list on the faculty dashboard of their students below the threshold, plus red percentages in reports
Continuous delivery	Deploy to a staging environment	On every merge the image is built and published to GitHub Container Registry; the demo machine runs it with docker compose. No automatic deployment
Quality gates	Tests with coverage, Bandit, SonarCloud	Also a lint and complexity check (ruff), a migration check and an 85% coverage floor. SonarCloud runs when the SONAR_TOKEN secret is set
Data model (Section 4.1.1)	Indexes on (student, subject, date) and (subject, date)	Two more: (subject, student, status) for report counts and (class, active) for class lists

Area	Version 1.0	As built, and why
Security details (Section 3.9)	HTTPS, hashing, lockout, role checks, audit	Also: sessions expire after 30 idle minutes, HTTPS-only cookies and safe headers in production, HSTS switchable once a trusted certificate is used

6.3 Division of Work as Carried Out

Table 1 was followed. Every member built and tested what it gives them; each also wrote non-functional test cases (TC-*-01 for SAM-NF-001 to 007) on one shared branch, one commit each, and every pull request was reviewed by the partner in Table 3 and merged by the repository owner.

Member	Built	Tested
Anmol Vyas (Developer)	Data model and seed data; Docker, Compose and Nginx; select class; mark and store attendance; modify with a reason; backup and restore; availability and data protection	UC-06 reports; subjects and assignments; TC-Perf-01, TC-Maint-01
Anupam Rajesh Tiwari (Developer)	Login and dashboards redirect; access control; lockout; students see only their own records; reports with CSV, PDF and print	UC-01 and UC-02; attendance modification; TC-UX-01
Anupriya (QA Lead)	CI pipeline; subjects, time slots and assignments; percentage; history; threshold and flags; report performance; quality gates and review rules	UC-04 and UC-05; TC-Load-01, TC-Reliability-01
Anusha G (Test Engineer)	Base templates and role dashboards; student and faculty management; audit module; lint, module-boundary checks and load data	UC-03 and UC-07; test data; RTM; test summary; TC-Recovery-01